



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційні систем та технологій

Лабораторна робота № 2
із дисципліни «Розробка мобільних застосунків під Android»
Тема: «ДОСЛІДЖЕННЯ РОБОТИ З КОМПОНЕНТОМ FRAGMENT»

Виконав

Студент групи ІА-31:

Губар Б. О.

Перевірив:

Орленко С. П.

Київ 2026

1. Мета: дослідити створення та взаємодію з компонентом Фрагмент (Fragment) компоненту Діяльність та набуті практичні навички з використання фрагментів для інтерфейсу користувача.

2. Завдання:

Написати програму під платформу Андроїд, яка має інтерфейс, побудований з декількох фрагментів згідно варіанту. Перший фрагмент представляє з себе форму для введення даних та кнопку підтвердження («ОК»), а інший фрагмент відображає результат взаємодії. Тобто другий фрагмент містить тестове поле з результатом та кнопкою «Cancel» (якщо згідно варіанту така існує, якщо ж за варіантом її немає – можете додати за власним бажанням), яка очищає або приховує (або видаляє) другий фрагмент та очищає форму введення з першого фрагменту. Зверніть увагу, що робота з фрагментами відбувається в рамках однієї Діяльності.

5.	Вікно замовлення квітів містить: текстове поле, дві групи опцій (колір, діапазон цін), тобто радіо-батонів та кнопку «ОК». Вивести інформацію щодо замовлення при натисканні на кнопку «ОК» у деяке текстове поле.
----	--

3. Хід роботи

Розмітка activity_main.xml: "Діяльність виступає виключно як контейнер. Інтерфейс складається з двох FrameLayout, куди динамічно підвантажуються InputFragment та ResultFragment.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="#F3F4F6">

    <FrameLayout
        android:id="@+id/inputFragmentContainer"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1" />

    <FrameLayout
        android:id="@+id/resultFragmentContainer"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

</LinearLayout>
```

Клас SharedViewModel.kt: "Для передачі та збереження стану між екранами використано архітектурний компонент ViewModel. Він містить MutableLiveData для зберігання тексту замовлення (orderResult) та подію для очищення форми (clearFormEvent).

```
package com.example.lab1_flowers

import androidx.lifecycle.MutableLiveData
import androidx.lifecycle.ViewModel
```

```

class SharedViewModel : ViewModel() {
    val orderResult = MutableLiveData<String?>()

    val clearFormEvent = MutableLiveData<Boolean>()
}

```

Клас MainActivity.kt: "Керує підключенням фрагментів за допомогою FragmentManager. Підписується на зміни у SharedViewModel. Якщо дані замовлення існують – відображає ResultFragment, якщо дані null – видаляє його.

```

package com.example.lab1_flowers

import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import androidx.lifecycle.ViewModelProvider

class MainActivity : AppCompatActivity() {
    private lateinit var viewModel: SharedViewModel

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        viewModel = ViewModelProvider(this)[SharedViewModel::class.java]

        if (savedInstanceState == null) {
            supportFragmentManager.beginTransaction()
                .replace(R.id.inputFragmentContainer, InputFragment())
                .commit()
        }

        viewModel.orderResult.observe(this) { result ->
            if (result != null) {
                supportFragmentManager.beginTransaction()
                    .replace(R.id.resultFragmentContainer, ResultFragment())
                    .commit()
            } else {
                val fragment =
                    supportFragmentManager.findFragmentById(R.id.resultFragmentContainer)
                if (fragment != null) {
                    supportFragmentManager.beginTransaction()
                        .remove(fragment)
                        .commit()
                }
            }
        }
    }
}

```

Клас InputFragment.kt: "Містить логіку валідації форми. При натисканні 'ОК' оновлює значення orderResult у спільній ViewModel. Також підписаний на clearFormEvent для очищення полів (editTextInput.text.clear(), radioGroup.clearCheck())."

```

package com.example.lab1_flowers

import android.os.Bundle
import android.view.LayoutInflater

```

```

import android.view.View
import android.view.ViewGroup
import android.widget.Button
import android.widget.EditText
import android.widget.RadioButton
import android.widget.RadioGroup
import android.widget.Toast
import androidx.fragment.app.Fragment
import androidx.lifecycle.ViewModelProvider

class InputFragment : Fragment() {
    private lateinit var viewModel: SharedViewModel

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        val view = inflater.inflate(R.layout.fragment_input, container, false)

        viewModel = ViewModelProvider(requireActivity())[SharedViewModel::class.java]

        val editTextInput = view.findViewById<EditText>(R.id.editTextInput)
        val radioGroupColor = view.findViewById<RadioGroup>(R.id.radioGroupColor)
        val radioGroupPrice = view.findViewById<RadioGroup>(R.id.radioGroupPrice)
        val buttonOk = view.findViewById<Button>(R.id.buttonOk)

        buttonOk.setOnClickListener {
            val name = editTextInput.text.toString().trim()
            val selectedColorId = radioGroupColor.checkedRadioButtonId
            val selectedPriceId = radioGroupPrice.checkedRadioButtonId

            if (name.isEmpty() || selectedColorId == -1 || selectedPriceId == -1) {
                Toast.makeText(requireContext(), "Будь ласка, заповніть всі поля та зробіть вибір!", Toast.LENGTH_SHORT).show()
            } else {
                val colorRadioButton = view.findViewById<RadioButton>(selectedColorId)
                val priceRadioButton = view.findViewById<RadioButton>(selectedPriceId)

                val result = "Замовлення оформлено!\nЗамовник: $name\nКолір квітів: ${colorRadioButton.text}\nКатеропія: ${priceRadioButton.text}"
                viewModel.orderResult.value = result
            }
        }

        viewModel.clearFormEvent.observe(viewLifecycleOwner) { shouldClear ->
            if (shouldClear) {
                editTextInput.text.clear()
                radioGroupColor.clearCheck()
                radioGroupPrice.clearCheck()
                viewModel.clearFormEvent.value = false
            }
        }

        return view
    }
}

```

Клас `ResultFragment.kt`: "Відображає дані зі `SharedViewModel`. Кнопка 'Cancel' встановлює значення замовлення у `null` (що змушує `Activity` видалити цей фрагмент) та відправляє сигнал `true` у `clearFormEvent` для першого фрагменту.

```
package com.example.lab1_flowers
```

```

import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Button
import android.widget.TextView
import androidx.fragment.app.Fragment
import androidx.lifecycle.ViewModelProvider

class ResultFragment : Fragment() {
    private lateinit var viewModel: SharedViewModel

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        val view = inflater.inflate(R.layout.fragment_result, container, false)

        viewModel = ViewModelProvider(requireActivity())[SharedViewModel::class.java]

        val textViewResult = view.findViewById<TextView>(R.id.textViewResult)
        val buttonCancel = view.findViewById<Button>(R.id.buttonCancel)

        viewModel.orderResult.observe(viewLifecycleOwner) { result ->
            textViewResult.text = result
        }

        buttonCancel.setOnClickListener {
            viewModel.orderResult.value = null
            viewModel.clearFormEvent.value = true
        }

        return view
    }
}

```

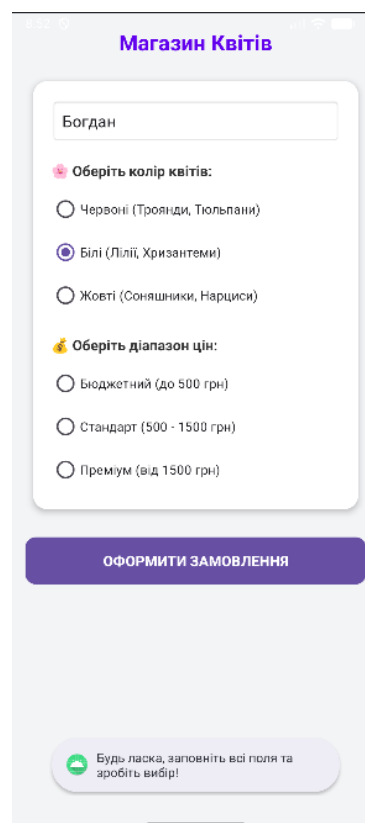


Рисунок 1 – Валідація даних

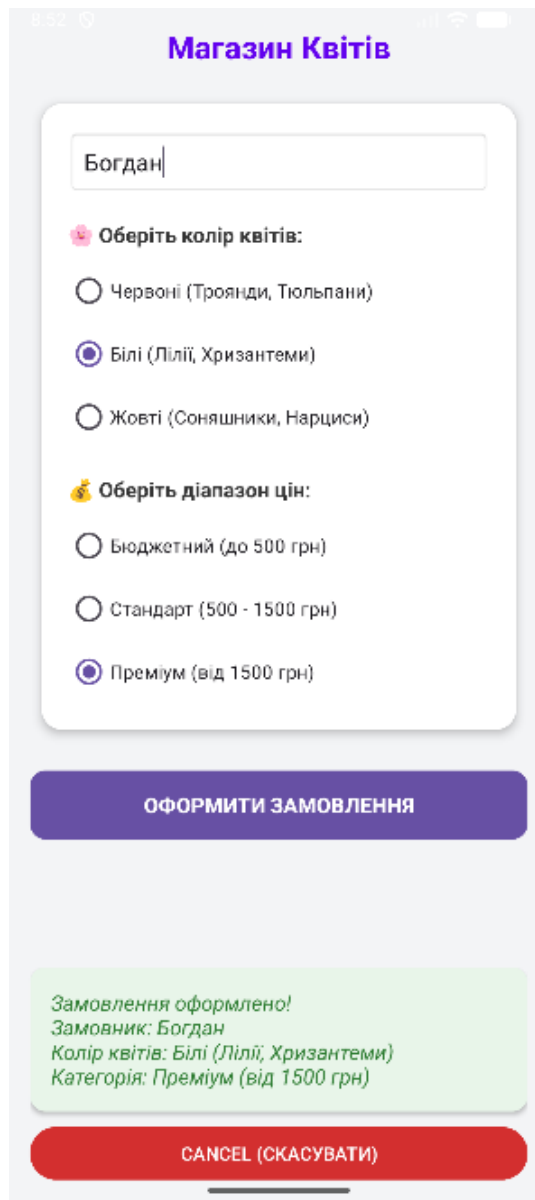


Рисунок 2 – Відображення результату (ResultFragment)

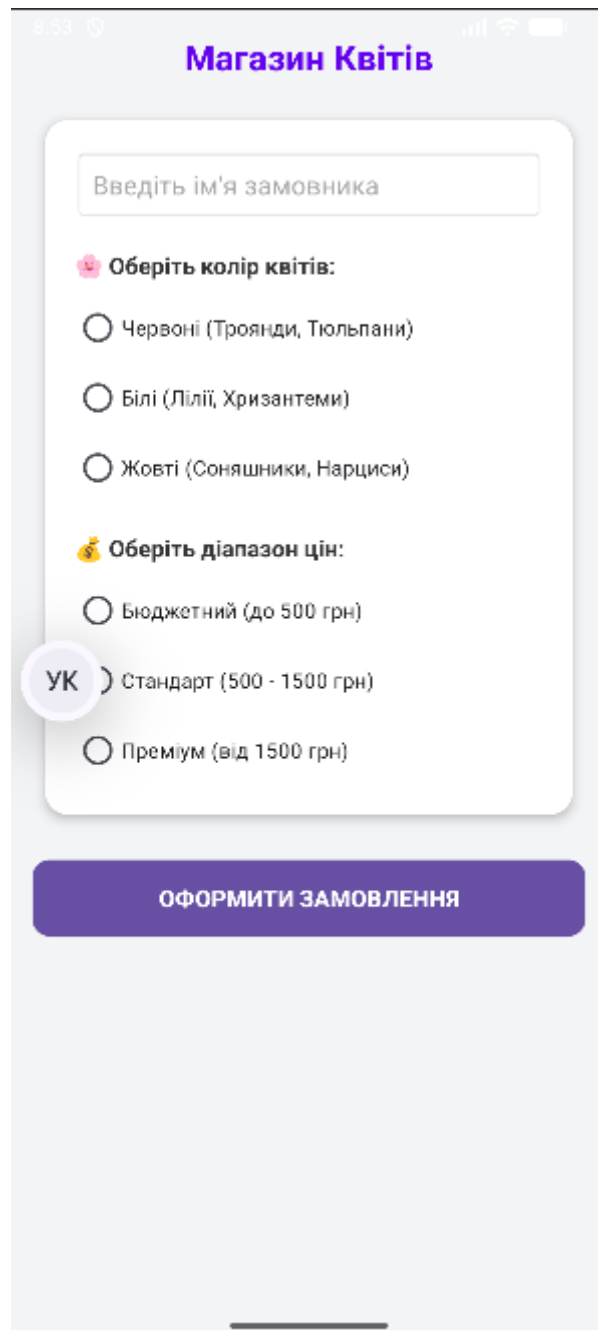


Рисунок 3 – Робота кнопки Cancel

Після натискання Cancel: нижній фрагмент зник, а текстове поле і радіо-кнопки у верхньому фрагменті знову порожні

4. Висновок

Під час виконання лабораторної роботи було досліджено життєвий цикл та принципи роботи з компонентом Fragment. Інтерфейс програми було успішно розділено на модульні складові: InputFragment (для збору даних) та ResultFragment (для відображення результату). Практично закріплено навички динамічного додавання та видалення фрагментів за допомогою FragmentManager та транзакцій. Передачу даних та управління станом інтерфейсу успішно реалізовано з використанням архітектурного компонента ViewModel.

5. Контрольні питання

- 1) Призначення та можливості компонента Фрагмент. Відповідь: Фрагмент — це модульна частина користувацького інтерфейсу в межах Activity. Він має власний життєвий цикл, отримує власні події введення, і його можна додавати або видаляти під час роботи Activity. Це дозволяє створювати гнучкі та адаптивні інтерфейси (наприклад, для телефонів та планшетів) і повторно використовувати код.
- 2) Життєвий цикл компонента Фрагмент. Відповідь: Життєвий цикл фрагмента тісно пов'язаний з життєвим циклом його Activity і включає специфічні методи: `onAttach()` (прив'язка до Activity), `onCreate()`, `onCreateView()` (створення UI), `onViewCreated()`, `onStart()`, `onResume()`. При знищенні викликаються: `onPause()`, `onStop()`, `onDestroyView()` (знищення UI), `onDestroy()`, `onDetach()` (відв'язка від Activity).
- 3) Способи створення компонента Фрагмент. Відповідь: Фрагменти можна створювати двома способами: статично (оголосивши тег `<androidx.fragment.app.FragmentContainerView>` у файлі XML-розмітки Activity) або динамічно (за допомогою програмного коду через `FragmentManager` під час виконання програми).
- 4) Опишіть способи управління компонентом Фрагмент. Відповідь: Управління фрагментами здійснюється через `FragmentManager`. Використовуються транзакції (`FragmentTransaction`), які дозволяють виконувати такі операції, як `add()` (додати), `replace()` (замінити), `remove()` (видалити), `hide()` (приховати) та `show()` (показати). Транзакції завершуються викликом методу `commit()`.
- 5) Опишіть способи взаємодії між Фрагментами. Відповідь: Сучасний та найбезпечніший спосіб — використання спільної `ViewModel`, прив'язаної до життєвого циклу хост-Activity (як реалізовано в цій роботі). Інші способи: використання `Fragment Result API` (`setFragmentResult` / `setFragmentResultListener`) або реалізація інтерфейсів зворотного виклику (`Callbacks`), які імплементує Activity.
- 6) Наведіть поняття системи, малої системи та мобільної платформи. Відповідь: Система — сукупність взаємопов'язаних елементів, що утворюють єдине ціле. Мала система (в контексті обчислень) — це зазвичай вбудована або мобільна система з обмеженими апаратними ресурсами та специфічним призначенням. Мобільна платформа — це апаратно-програмний комплекс (ОС, API, інструменти), призначений для розробки та запуску застосунків на мобільних пристроях.
- 7) Опишіть типи мобільних застосунків. Відповідь: 1. Нативні — розроблені під конкретну платформу з використанням її офіційних інструментів (Kotlin/Java для Android, Swift для iOS). 2. Веб-застосунки — адаптивні сайти, що працюють через браузер мобільного пристрою. 3. Гібридні — веб-застосунки, загорнуті в нативний контейнер (наприклад, `WebView`). 4. Кросплатформні — написані єдиним кодом, що компілюється під різні платформи (Flutter, React Native).

- 8) Наведіть класифікацію та характеристику середовищ розробки мобільних застосунків. Відповідь: Середовища поділяються на Нативні IDE (Android Studio для Android — надає повний доступ до SDK та профілювання; Xcode для iOS), Кросплатформні IDE/Редактори (VS Code, IntelliJ IDEA) та Хмарні/Low-code платформи для швидкого прототипування без глибоких знань коду.
- 9) Наведіть класифікацію та характеристику мобільних платформ. Відповідь: Основний поділ відбувається за ступенем відкритості та архітектурою. Відкриті платформи (Android) базуються на ядрі Linux, мають відкритий вихідний код, широкі можливості кастомізації та велике різноманіття апаратного забезпечення. Закриті платформи (iOS) базуються на UNIX, мають закритий код, жорстко контролюються виробником (Apple) та характеризуються високим рівнем безпеки й оптимізації під конкретне залізо.